

Amit Bal

asb2332

Github link: <https://github.com/ASB1127/A3-Transformer.git>

Tiny Transformer for Next-Token Prediction

Introduction

Transformer models have become central to modern natural language processing because they provide a flexible way to model relationships between tokens in a sequence. Instead of processing text strictly one step at a time, a Transformer uses self-attention to let each token compare itself with other tokens in the context. This makes the architecture especially useful for language modeling, where the model must use previous words or subword units to predict what should come next. The goal of this assignment was to build a small Transformer from scratch and use it to better understand how the architecture works.

In this project, I trained a Tiny Transformer language model on the Tiny Shakespeare corpus for next-token prediction. The text was first converted into subword tokens using Byte Pair Encoding, then arranged into fixed-length input and target sequences where the target is shifted one position ahead of the input. The model itself includes token embeddings, sinusoidal positional encodings, masked multi-head self-attention, residual connections, RMSNorm, and a feed-forward network. After training, I evaluated the model using validation perplexity and visualized its behavior through loss curves and attention heatmaps. These visualizations are used later in the report to connect the model's quantitative performance with the way its attention heads distribute focus across earlier tokens in the sequence.

Data Preparation

The dataset used for this project was the Tiny Shakespeare corpus, a small text dataset made up of Shakespeare-style dialogue. Since the task was next-token prediction, the text first had to be converted into a numerical format that the model could process. I used Byte Pair Encoding as the tokenizer with a vocabulary size of 500. This gave the model subword-level tokens instead of individual characters, which made the representation more meaningful while still keeping the vocabulary small enough for a lightweight experiment.

After tokenization, the full token sequence was split into training and validation portions using an 80/20 split. I made this split before creating the overlapping input windows so that the validation examples would come from a separate part of the token stream. Each example used a fixed sequence length of 50 tokens. The input sequence contained 50 consecutive tokens, and the target sequence was the same sequence shifted one position to the right. This means the model learned to predict the next token at every position in the sequence. For example, if the input tokens are $[x_1, x_2, x_3, \dots]$, the target tokens are $[x_2, x_3, x_4, \dots]$. This setup matches the causal language modeling objective used by larger autoregressive language models.

Model Architecture

The model begins by converting each input token ID into a learned embedding vector of size 128. These embeddings represent the subword tokens in a continuous space that can be updated during training. Before the sequence enters the Transformer blocks, I add sinusoidal positional encodings to the embeddings so the model has information about token order.

Since self-attention does not naturally know the order of tokens, positional encodings are necessary for distinguishing where each token appears in the sequence. Without this added position information, the model would see the same set of token embeddings but would not have a direct way to tell whether a token appeared early or late in the context. This matters for language modeling because word order affects meaning and changes which next tokens are likely.

The main body of the model consists of two Transformer blocks. Each block contains a masked multi-head self-attention layer followed by a feed-forward network. I used four attention heads, which means the hidden representation is split into four smaller parts so the model can learn multiple attention patterns in parallel. In the self-attention layer, each token produces a query, key, and value vector. Attention scores are computed by comparing queries against keys, scaling the scores, applying a causal mask, and then using softmax to produce attention weights. The causal mask prevents a token from attending to future positions, so the model can only use the current token and previous tokens when making predictions.

Each Transformer block also uses residual connections and RMSNorm. The residual connections add the attention output back to the original representation, and later add the feed-forward output back as well. This helps preserve information across layers and makes training more stable. RMSNorm is applied before the attention and feed-forward sublayers, which helps control the scale of the hidden activations. The feed-forward network expands the hidden size from 128 to 512, applies a GELU activation, and projects it back down to 128. After the two Transformer blocks, a final RMSNorm and linear output layer map the hidden states to vocabulary-sized logits for next-token prediction.

Training Setup and Evaluation

The model was trained with cross-entropy loss, which measures how close its next-token predictions are to the correct targets. Since every input sequence has a shifted target sequence, the loss is computed across all token positions in the batch rather than only at the final token. This allows the model to learn from many next-token prediction examples during each training step.

I trained the model for five epochs with a batch size of 64 and a learning rate of 0.0001 using the Adam optimizer. The training data was shuffled using a fixed random seed, while the validation data was kept in order for evaluation. I also set random seeds for Python, NumPy, and PyTorch to make the experiment more reproducible. During validation, I used `torch.no_grad()` so that gradients were not stored, which reduced memory use and made evaluation more efficient.

The main evaluation metric was validation perplexity. Perplexity is computed by exponentiating the validation cross-entropy loss, so a lower perplexity means the model assigns higher probability to the correct next tokens. In language modeling, perplexity is useful because it summarizes how uncertain the model is when predicting unseen text. I also tracked training and validation loss across epochs to check whether the model was learning and whether the validation loss followed the same general trend as the training loss.

Results

The training and validation loss curves show that the model learned the next-token prediction task over the five training epochs. At the beginning of training, the model's predictions were still close to random over the small vocabulary, so the cross-entropy loss was high. As training continued, both the training loss and validation loss decreased steadily. This means the model was improving not only on the training sequences, but also on the held-out validation portion of the corpus.

The validation loss followed the training loss closely, which is a useful sign for this experiment. If the training loss decreased while the validation loss increased, that would suggest overfitting. In this run, the validation curve moved downward with the training curve, so the model was still learning patterns that generalized to unseen validation sequences. Since the model was only trained for five epochs, the curves do not show a full plateau. Additional training might improve performance for a little longer, but overfitting would become a concern because Tiny Shakespeare is a small dataset. Even though the Transformer used here is lightweight, training it for too many epochs could cause it to memorize patterns from the training split instead of learning patterns that generalize well to validation text.

The final validation loss was 4.8309, which corresponds to a validation perplexity of 125.3186. Perplexity can be interpreted as a measure of how uncertain the model is when predicting the next token. A lower perplexity means the model is assigning more probability to the correct next tokens. Although this perplexity is not low compared with larger language models, it is reasonable for a small Transformer trained on a limited dataset with a small hidden size and only two Transformer blocks. The main purpose of this assignment was to verify that the Transformer implementation could learn the structure of the text and produce meaningful internal attention patterns, rather than to maximize language modeling performance.

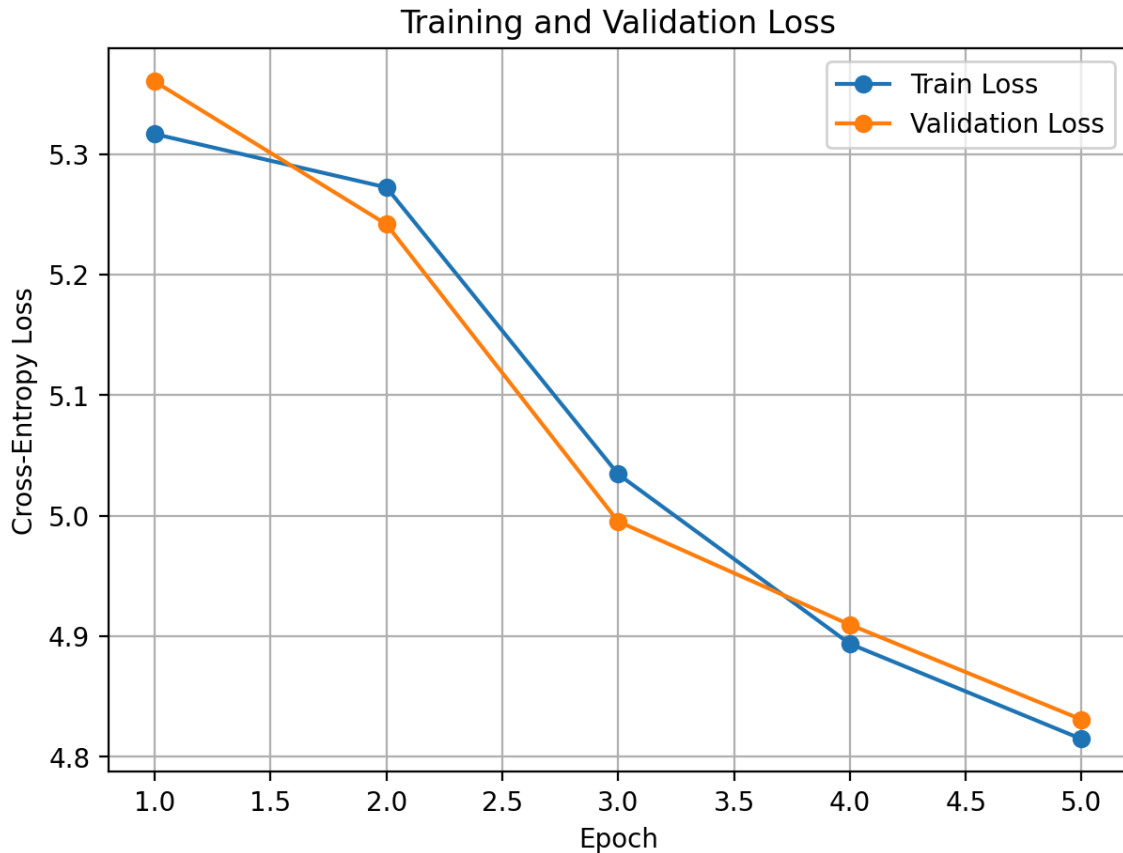


Figure 1. Training and validation loss over five epochs.

Attention Visualization and Interpretation

To better understand what the model was doing internally, I visualized attention weights from the trained Transformer. The heatmaps show attention patterns for two heads from each of the two Transformer blocks. In each heatmap, the rows represent query tokens and the columns represent key tokens. A brighter value means that the query token is placing more attention on that key token when forming its representation.

The most obvious pattern in the heatmaps is the dark purple upper-right triangle caused by the causal mask. In the color map, dark purple represents attention weights that are zero or close to zero. These positions correspond to future tokens, which the model is not allowed to use when predicting the next token. The brighter lower-left region shows the current and previous tokens that are available for attention. This confirms that the model is not using future information during next-token prediction, which is important because the task should only depend on the current and previous context.

Another pattern is that attention is strongest near the diagonal, especially for nearby previous tokens. This means the model often relies more heavily on recent context than on tokens farther back in the sequence. This makes sense for a short-context language

modeling task because nearby subword tokens often carry useful information about the next token. For example, parts of a word, punctuation, or recently seen words can strongly influence what comes next. The model still has access to the full 50-token context, but the heatmaps suggest that much of the attention mass is concentrated on the most recent portion of the sequence.

Across the plotted heads and layers, the attention maps look broadly similar rather than highly specialized. This may be because the model is small and was trained for only five epochs, so the heads may not have had enough capacity or training time to develop very distinct roles. It is also possible that the selected validation sequence does not require very different long-range attention behavior. Still, the visualization is useful because it shows that the attention mechanism and causal masking are functioning as intended. The model is building each token representation by looking backward through the sequence, with a stronger focus on nearby tokens.

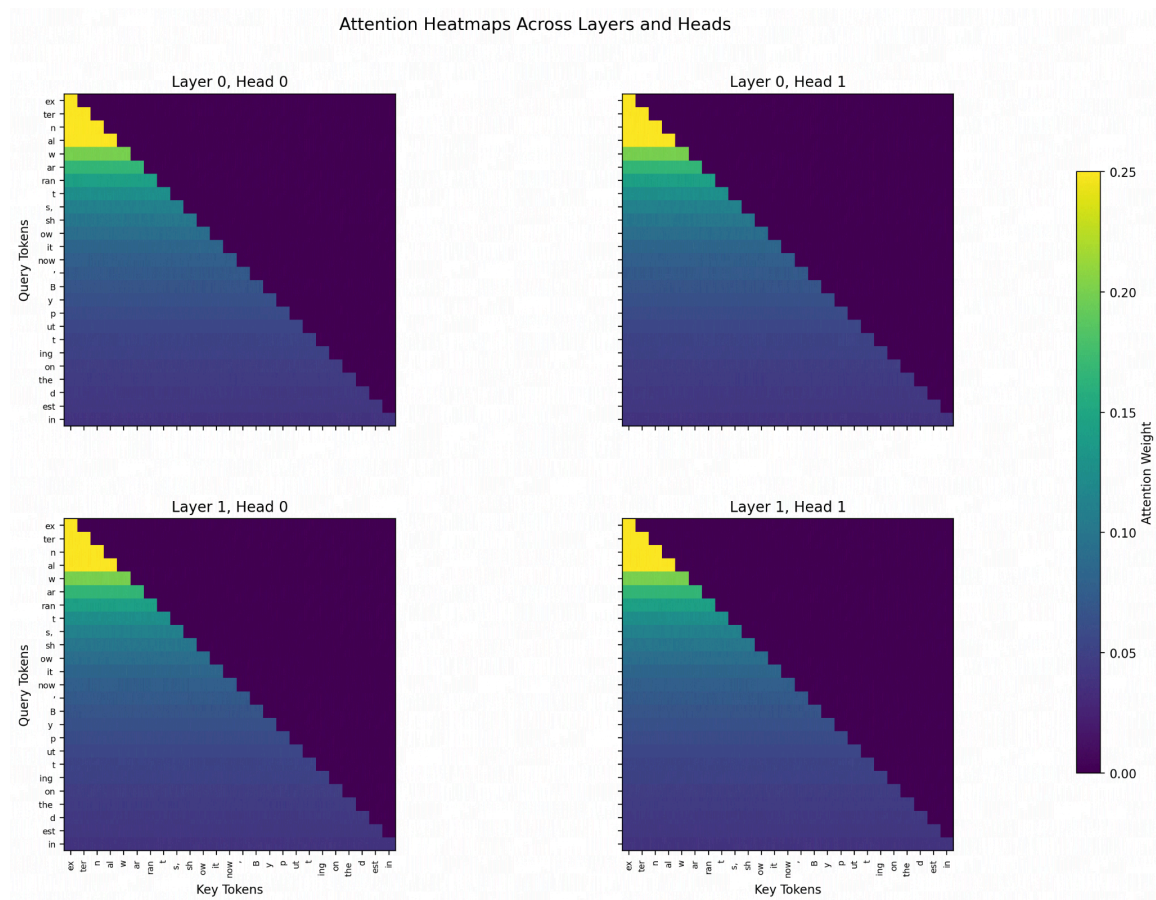


Figure 2. Attention heatmaps for two heads in each Transformer block.

Discussion and Reflection

The hyperparameters that seemed most important for stability were the learning rate, context length, and model size. I used a learning rate of 0.0001, which was conservative

enough for the loss to decrease smoothly over the five epochs. A larger learning rate might have trained faster, but it could also have made the loss less stable, especially because the attention and Transformer block were implemented from scratch. The context length of 50 tokens gave the model enough local context to learn short phrases without making the attention matrices too large. Since attention cost grows quickly as sequence length increases, a much longer context would increase memory use and runtime. The hidden size of 128 and two Transformer blocks also kept the model lightweight while still giving it enough capacity to learn meaningful patterns.

Positional encodings also played an important role. Self-attention compares tokens to one another, but by itself it does not know where tokens occur in the sequence. Without positional encodings, the model would have token identity information but much weaker information about token order. For language modeling, this would be a serious limitation because the same words can mean different things depending on their order. Adding sinusoidal positional encodings allowed the model to combine token meaning with position information before applying attention.

I only visualized the final trained model's attention maps, so I cannot make a strong claim about exactly how attention evolved across epochs. However, the loss curves show that the model's predictions improved steadily during training. If I extended this experiment, I would save attention maps after each epoch and compare them over time. That would make it possible to see whether the model first learns simple causal or local patterns and later develops more specialized attention heads. In the current results, the heads look broadly similar, so this would be an interesting direction for deeper analysis.

AI Tool Usage Disclosure

I used AI assistance as a support tool while completing this assignment, but I wrote most of the core implementation myself. AI assistance was used to clarify specific implementation choices, including the multi-head attention projection design, the shape assertion for the number of attention heads, and the overall TinyTransformerLM structure. It also helped me compare my custom attention module against `torch.nn.MultiheadAttention` and improve the attention heatmap so it was reproducible and showed multiple layers and heads.

AI assistance was also used to help prepare supporting repository files, including `requirements.txt`, `README.md`, `.gitignore`, and a Python script version of the notebook, and to draft parts of this report. My own contributions included writing most of the Transformer code, asking design questions, choosing which suggestions to use, running the notebook, checking the numerical results, reviewing the generated plots, and deciding what analysis was accurate enough to include. The final code, experiments, figures, and explanations reflect my understanding of the Transformer components implemented in this project.